

Raport Observability & Data Science

Analiza logów systemu rozproszonego

Automatyczna analiza (wydruk z programu) + interpretacja ekspercka

1 Zakres analizy

- Liczba eventów: 1 683 588
- Liczba transakcji (TransactionId): 246 376
- Liczba korelacji (CorrelationId): 246 376
- Mikroserwisy: 6 (ApiGateway, OrderService, PaymentService, NotificationService, AuthService, InventoryService)

1.1 Rozkład scenariuszy

Obciążenie scenariuszami jest równomierne (ok. 160–166 tys. zdarzeń na większość scenariuszy), co sugeruje, że problemy nie wynikają wyłącznie z jednego przypadku użycia. Jednocześnie wyróżniają się scenariusze z podwyższoną liczbą anomalii: **BulkImport**, **CheckoutWithTransfer**, **PasswordReset**.

2 Szybka ocena kondycji systemu

2.1 Poziom ważności zdarzeń (Priority)

- info: 1 652 724 (dominująca większość) **OK**
- warn: 21 560 (sygnały degradacji) **UWAGA**
- crit: 8 373 oraz err: 931 (incydenty) **UWAGA**

2.2 Kody zdarzeń (EventCode)

- 200: 1 612 428 (zdecydowana większość) **OK**
- 500/501: 8 514 (ok. 0.51% całości) **UWAGA**
- 429: 20 991 (rate limiting / przeciążenie) **UWAGA**

2.3 Wniosek ogólny

System w ujęciu całościowym jest **stabilny** (dominacja 200/info), ale występują **epizody przeciążeń** (429, spike-y CPU/QPS/latencji) oraz **rzadkie, ale istotne awarie 5xx**.

3 Charakterystyka metryk (z wykresów i statystyk)

3.1 LatencyMs: typowy szybki system + długi ogon

Rozkład opóźnień jest silnie skośny:

- większość zdarzeń w przedziale **20–518 ms** (dominujący bin),
- występuje **długi ogon** do ok. **10 000 ms** (incydentalne, ale kosztowne dla UX/SLA),
- w oknach trendu widać **wyraźny epizod podniesionej mediany** (przykład dla OrderService).

3.2 CPU usage: rozkład wielomodalny

Z histogramu CPU widać co najmniej dwa reżimy pracy:

- normalny tryb (duża masa obserwacji w średnich wartościach),
- okresy wysokiego CPU (osobna “górką” w wysokich wartościach), spójne z detekcją spike CPU.

3.3 DiskQueueLength: dominują wartości niskie, ale występują wysokie kolejki

Rozkład jest skoncentrowany przy niskich wartościach, jednak sporadycznie pojawiają się bardzo wysokie kolejki dyskowe (do kilkuset), co jest silnym wskaźnikiem:

- ograniczeń I/O,
- blokad (np. DB, filesystem),
- backpressure i kumulowania requestów.

3.4 LocalQps: typowo niski/średni, z okresami burstów

Większość obserwacji ma umiarkowany QPS, ale występują **bursty** (odchylenia wysokie), które korelują ze spike-ami i degradacją latencji.

4 Spike detection: skala i interpretacja

4.1 Skala spike-ów (łącznie)

- **Latency spikes:** 89 022
- **CPU spikes:** 88 200
- **QPS spikes:** 71 717

Suma (liczona per typ) to 248 939, co należy rozumieć jako “liczbę sygnałów”, a niekoniecznie unikalnych zdarzeń (te same eventy mogą spełniać kilka kryteriów).

4.2 Spike per serwis: symetria

Każdy serwis ma bardzo zbliżoną liczbę spike-ów (ok. 41–42 tys.). To ważna obserwacja: **nie wygląda to na jeden wadliwy mikroserwis**, tylko raczej na:

- wspólne źródło przeciążenia (ruch, zależność wspólna, baza, sieć),
- efekt kaskadowy (jeden problem → kolejne serwisy “łapią” spike-y),
- ograniczenia infrastrukturalne wspólne dla klastra.

4.3 Jakość detekcji spike

Wyniki jakościowe są wysokie (szczególnie CPU i QPS):

- Latency: Precision 77.77%, Recall 100%
- CPU: Precision 92.03%, Recall 100%
- QPS: Precision 100%, Recall 100%

Interpretacja: system **nie przegapia** spike-ów, ale latencja generuje pewną liczbę false positive (typowe przy długim ogonie).

5 Trend analysis: przykład OrderService

5.1 Wykryte trendy

Dla OrderService:

- trend window events: 23 874 (2 przedziały),
- wykryte buckety trendu: 295,
- Precision: 0.973, Recall: 0.420, F1: 0.587.

Wniosek: mechanizm trendów jest **ostrożny** (wysoka precyzja), ale **nie wychwytuje wszystkich** epizodów trendowych (niższy recall). Operacyjnie oznacza to mniej fałszywych alarmów, ale ryzyko przeoczenia części pogorszeń.

6 Failure detection: co faktycznie znaczy “awaria” w systemie

6.1 Podejście 1: kody 5xx (EventCode 500/501)

- Precision: 47.9%
- Recall: 99.98%
- F1: 64.8%

To podejście łapie praktycznie wszystkie awarie, ale ma dużo false positive (część kandydatów ma symptomy awarii bez kodu 5xx).

6.2 Podejście 2: flaga FailureWindow

- Precision: 95.6%
- Recall: 79.1%
- F1: 86.6%

To podejście jest bardziej przydatne do alertowania on-call: mniej szumu, sensowny kompromis.

6.3 Propagacja awarii po CorrelationId (kaskadowość)

Gdy pojawia się pierwszy failure w systemie X, w tej samej korelacji **ok. 58–63%** zdarzeń w innych systemach staje się kandydatami awarii. To wskazuje na **ryzyko cascading failure** i potrzebę izolacji błędów między usługami.

7 Anomaly detection (Isolation Forest): co mówi model

7.1 Wynik

- wykryto 852 anomalie (0.0506%),
- Precision 0.488, Recall 0.527, F1 0.507.

Wniosek: model jest **średnio zgodny** z etykietą `IsAnomaly`. To może oznaczać:

- etykieta jest trudna / niedoskonała (niepełna definicja anomalii),
- parametry modelu (np. `contamination`) są zbyt agresywne lub zbyt zachowawcze,
- potrzebne są cechy kontekstowe (np. `endpoint`, typ operacji, zależność downstream).

7.2 Top scenariusze z anomaliami

- `BulkImport` (92)
- `CheckoutWithTransfer` (90)
- `PasswordReset` (89)

Dominujące powody: **latencja + dysk + sieć**.

8 K-means: segmentacja zachowań systemu

8.1 Najważniejszy wniosek (K=3)

Istnieje wyraźny klaster przeciążeniowy (ok. 93 566 zdarzeń):

- **Latency**: 607.6 ms
- **CPU**: 86.8%
- **DiskQueueLength**: 85.6
- **NetworkErrors**: 11.6
- **LocalQps**: 238

oraz bardzo wysokie udziały spike-ów (CPU/Latency/QPS).

To praktycznie podpis: **przeciążenie + degradacja I/O/sieci**.

8.2 Ekstremalny klaster (K=8, klaster 6)

Jest mały (592 zdarzenia), ale krytyczny:

- Latency ok. 6.97 s,
- `DiskQueueLength` ok. 160,
- `NetworkErrors` ok. 27,
- 100% oznaczonych jako anomalia.

To są epizody, które użytkownik **odczuwa jako “system przestał działać”**.

9 Root Cause Analysis: przykład incydentu

Dla korelacji b5145... w scenariuszu PasswordReset:

- ścieżka: OrderService → PaymentService → NotificationService → ApiGateway → InventoryService,
- **pierwsze źródło problemu: PaymentService** (krok 2),
- przyczyny: ekstremalny spike latencji, wzrost błędów sieci, nietypowy rozmiar odpowiedzi,
- ofiary przeciążenia: NotificationService, ApiGateway, InventoryService.

To wspiera tezę, że **awarie są kaskadowe**, a izolacja błędów między usługami jest kluczowa.

10 Proponowane środki zaradcze

10.1 Krótkoterminowe (0–2 tygodnie) – szybkie usprawnienia

- **Alerty na p95/p99 latencji** per serwis i per scenariusz (nie tylko średnia/mediana).
- **Alerty na DiskQueueLength** (np. progi 50/100) oraz korelacja z latencją.
- **Alerty na NetworkErrors i Retries** (ryzyko retry-storm).
- Przegląd **timeoutów** i **connection pool** w PaymentService (i usługach downstream).
- Limitowanie rozmiaru odpowiedzi (sanity-check na ResponseSizeBytes), włączenie paginacji / chunkingu.

10.2 Średnioterminowe (2–8 tygodni) – stabilizacja pod obciążeniem

- **Rate limiting** i **load shedding** (szczególnie dla BulkImport/CheckoutWithTransfer).
- **Circuit breaker + fallback** między usługami (redukcja cascading failure).
- **Autoscaling** oparty o QPS/latencję, nie tylko CPU.
- Osobne kolejki/limity dla zadań ciężkich (BulkImport, reporty) versus ruch interaktywny.

10.3 Długoterminowe (2–6 miesięcy) – inwestycje i architektura

- Inwestycja w **observability end-to-end**: tracing per korelacja, SLO per scenariusz, dashboardy p95/p99.
- Wydzielenie operacji ciężkich do **asynchronicznego pipeline** (kolejki, worker-y).
- Praca nad **wydajnością I/O** (DB/Storage): indeksy, limity, archiwizacja, cache.
- **Chaos / load testing** cyklicznie (symulacja burstów, awarii sieci, spadków I/O).
- Rewizja architektury PaymentService: krytyczna zależność, częsty root cause.

11 Executive Summary (dla osoby nietechnicznej)

System w większości działa poprawnie, ale ma okresowe momenty przeciążenia, które powodują wolne działanie (opóźnienia) oraz sporadyczne błędy. Problemy nie dotyczą jednego modułu — pojawiają się podobnie w wielu usługach i potrafią “rozlewać się” na inne części systemu (efekt domina).

Największe ryzyko to gwałtowne skoki obciążenia oraz problemy z dostępem do zasobów (dysk/sieć), które w krytycznych chwilach powodują kilkusekundowe czasy odpowiedzi. To bezpośrednio wpływa na doświadczenie użytkownika i może podnosić liczbę porzuconych operacji.

Co zmienić od razu

- Ustawić monitoring na “czasy odpowiedzi w gorszych przypadkach” (p95/p99), nie tylko średnią.
- Wprowadzić automatyczne alarmy na wzrost kolejek dyskowych i błędów sieci.

W co zainwestować

- **Autoscaling** (skalowanie automatyczne przy wzroście ruchu).
- **Mechanizmy ochronne** między usługami (circuit breaker, rate limiting), aby awaria jednego elementu nie psuła całego systemu.
- **Lepszą obserwowalność** (tracing i dashboardy SLA/SLO), żeby szybciej znajdować przyczyny problemów.
- **Testy obciążeniowe** i planowanie pojemności (capacity planning) przed wzrostem ruchu.